Lab 3.3 – Poison Messages and Message Encoding

Module M9 · Estimated time: 60–75 minutes

Scenario

A Function App consumes orders from Azure Queue Storage. Some messages fail because the producer sends an incompatible message format. Other messages contain malformed JSON and repeatedly fail processing.

You must identify the encoding issue, correct it, configure retry behavior and handle messages that exceed the retry limit.

Step 1 – Prepare the Function project

Use the existing `contoso-fn` project or your established Day 3 Function project.

Install the Queue extension:

```
dotnet add package
Microsoft.Azure.Functions.Worker.Extensions.Storage.Que
ues
```

Confirm the package:

```
dotnet list package
```

```
PS C:\Users\LabsKraft\Day3-QueueLab\function-app> dotnet list package
Restore complete (2.0s)

Build succeeded in 3.1s
Project 'PoisonMessages' has the following package references
   [net10.0]:
   Top-level Package                                          Requested   Resolved
   > Microsoft.Azure.Functions.Worker                         2.0.0       2.0.0
   > Microsoft.Azure.Functions.Worker.Extensions.Storage.Queues 5.5.5      5.5.5
   > Microsoft.Azure.Functions.Worker.Sdk                     2.0.0       2.0.0
   > Microsoft.Extensions.Hosting                             10.0.0      10.0.0

PS C:\Users\LabsKraft\Day3-QueueLab\function-app>
```

Step 2 – Configure `host.json`

Set the Queue extension configuration:

```
{
  "version": "2.0",
  "extensions": {
    "queues": {
      "batchSize": 1,
      "maxDequeueCount": 3,
      "visibilityTimeout": "00:00:10",
      "maxPollingInterval": "00:01:00",
      "messageEncoding": "base64"
    }
  }
}
```

For this exercise:

- `batchSize = 1` makes retry behavior easier to observe.
- `maxDequeueCount = 3` limits the message to three processing attempts.

- `visibilityTimeout = 10 seconds` makes the retry cycle easier to demonstrate.
- `messageEncoding = base64` means the Functions Queue extension expects Base64-encoded messages.



Step 3 – Create the Queue-triggered Function

Create `OrderQueueProcessor.cs`:

```
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;
using System.Text.Json;

namespace ContosoFn;

public class OrderQueueProcessor
{
    private readonly ILogger<OrderQueueProcessor>
_logger;
```

```
    public OrderQueueProcessor(
        ILogger<OrderQueueProcessor> logger)
    {
        _logger = logger;
    }

    [Function("OrderQueueProcessor")]
    public void Run(
        [QueueTrigger(
            "orders",
            Connection = "AzureWebJobsStorage")] string
message)
    {
        _logger.LogInformation(
            "Received order message: {Message}",
message);

        using JsonDocument document =
JsonDocument.Parse(message);

        string orderId = document.RootElement
            .GetProperty("orderId").GetString()!;

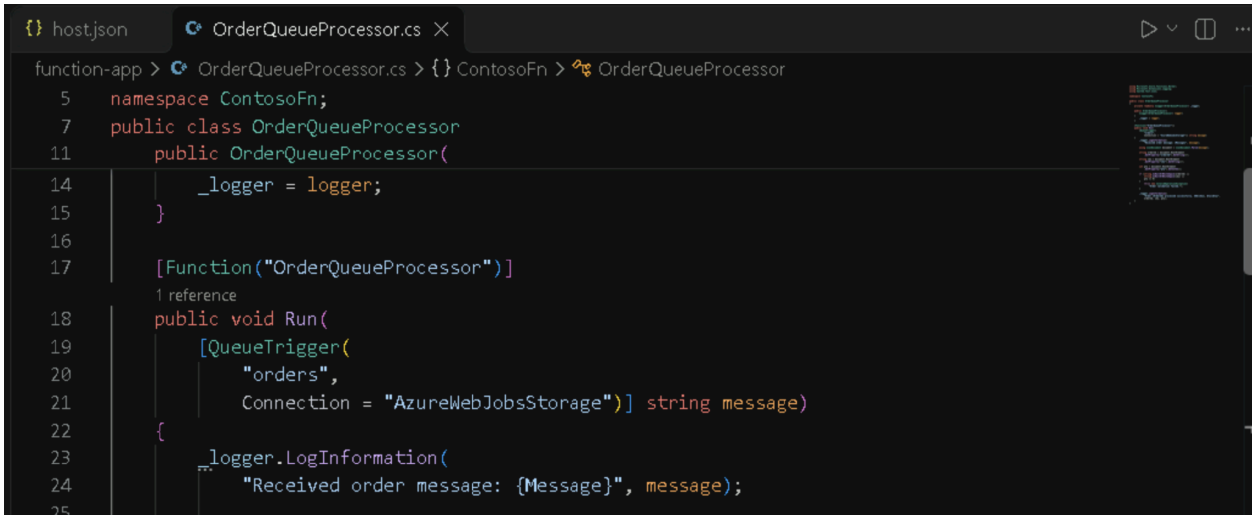
        string sku = document.RootElement
            .GetProperty("sku").GetString()!;

        int qty = document.RootElement
            .GetProperty("qty").GetInt32();
```

```
        if (string.IsNullOrWhiteSpace(orderId) ||
            string.IsNullOrWhiteSpace(sku) ||
            qty <= 0)
        {
            throw new InvalidOperationException(
                "Order validation failed.");
        }

        _logger.LogInformation(
            "Order {OrderId} processed successfully.
SKU={Sku}, Qty={Qty}",
            orderId, sku, qty);
    }
}
```

This Function intentionally throws an exception when the message is invalid. The Functions host will retry the message according to the Queue configuration.



Step 4 – Run the Function locally

For local testing, ensure `local.settings.json` uses Azurite:

```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage":
"UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated"
  }
}
```

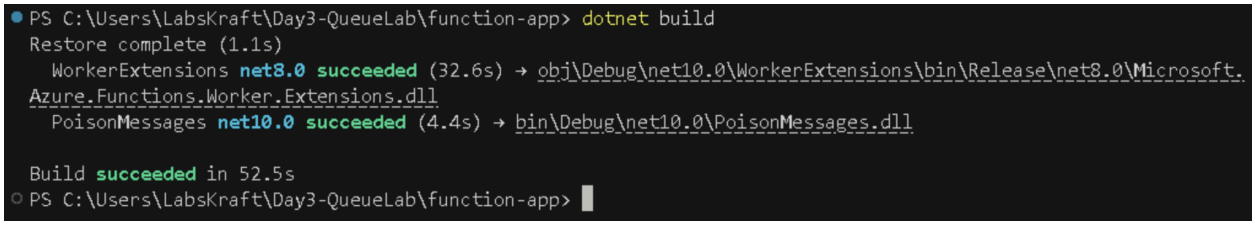
Start Azurite and create the `orders` queue in the local emulator using Storage Explorer.



Build and start the Function:

```
dotnet build
func start
```

For this lab's local producer tests, use the local Azurite connection string. Do not replace the Azure Function App's deployed storage configuration with this local setting.



```
PS C:\Users\LabsKraft\Day3-QueueLab\function-app> func start
Core Tools Version:       4.16.0+ff7b011b5f26de583a53044c7fdd3b26b7229c1d (64-bit)
Function Runtime Version: 4.1054.250.26428

[2026-11-05T17:35:42.655Z] Found C:\Users\LabsKraft\Day3-QueueLab\function-app\PoisonMessages.csproj. Using fo
r user secrets file configuration.
[2026-11-05T17:35:47.478Z] Worker process started and initialized.

Functions:

        OrderQueueProcessor: queueTrigger

        OrdersPoisonHandler: queueTrigger

        ProcessOrder: queueTrigger

For detailed output, run func with --verbose flag.
[2026-11-05T17:35:52.630Z] Host lock lease acquired by instance ID '00000000000000000000000C617C060'.
```

Step 5 – Create the C# producer project

Create a separate console project:

```
mkdir Day3-QueueProducer
cd Day3-QueueProducer

dotnet new console --framework net10.0
dotnet add package Azure.Storage.Queues
```

Replace `Program.cs`:

```
using Azure.Storage.Queues;
using Azure.Storage.Queues.Models;
using System.Text.Json;

const string connectionString =
    "UseDevelopmentStorage=true";

const string queueName = "orders";
```

```
var queue = new QueueClient(
    connectionString,
    queueName);

await queue.CreateIfNotExistsAsync();

string mode = args.FirstOrDefault() ?? "plain";

var order = new
{
    orderId = "D3-ENC-001",
    sku = "SKU-501",
    qty = 2
};

string json = JsonSerializer.Serialize(order);

if (mode == "plain")
{
    await queue.SendMessageAsync(json);
    Console.WriteLine("Plain-text message sent.");
}
else if (mode == "base64")
{
    var encodedQueue = new QueueClient(
        connectionString,
        queueName,
        new QueueClientOptions
        {
```

```
            MessageEncoding =
QueueMessageEncoding.Base64
        });

    await encodedQueue.SendMessageAsync(json);
    Console.WriteLine("Base64-encoded message sent.");
}
else if (mode == "malformed")
{
    var encodedQueue = new QueueClient(
        connectionString,
        queueName,
        new QueueClientOptions
        {
            MessageEncoding =
QueueMessageEncoding.Base64
        });

    await encodedQueue.SendMessageAsync(
        "{ this is not valid JSON ");

    Console.WriteLine("Malformed message sent.");
}
```

Step 6 – Reproduce the encoding failure

Run the producer:

```
dotnet run -- plain
```

Observe the Function logs.

Expected: the message fails because the Functions Queue extension is configured to expect Base64 encoding.

```
PS C:\Users\LabsKraft\Day3-QueueLab\Day3-QueueProducer> dotnet run -- plain
Plain-text message sent.
PS C:\Users\LabsKraft\Day3-QueueLab\Day3-QueueProducer>
```

```
Functions:

        OrderQueueProcessor: queueTrigger

        OrdersPoisonHandler: queueTrigger

        ProcessOrder: queueTrigger

For detailed output, run func with --verbose flag.
[2026-11-05T17:54:08.114Z] Message has reached MaxDequeueCount of 3. Moving message to queue '
orders-poison'.
[2026-11-05T17:54:12.459Z] Host lock lease acquired by instance ID '000000000000000000000000C6
17C060'.
```

This is the planted defect. Do not change `host.json` yet.

Step 7 – Fix the producer

Send a correctly encoded message:

```
dotnet run -- base64
```

Expected:

- The Function receives the message.
- JSON parsing succeeds.
- Order validation succeeds.
- The Function logs successful processing.

```
PS C:\Users\LabsKraft\Day3-QueueLab\Day3-QueueProducer> dotnet run -- base64
Base64-encoded message sent.
PS C:\Users\LabsKraft\Day3-QueueLab\Day3-QueueProducer>
```

```
[2026-11-05T17:54:12.459Z] Host lock lease acquired by instance ID '00000000000000000000000000C617C060'.
[2026-11-05T17:56:10.608Z] Executing 'Functions.OrderQueueProcessor' (Reason='New queue message detected on 'orders'.', Id=1d51098d-aaa0-4115-92d1-75349215914d)
[2026-11-05T17:56:10.612Z] Trigger Details: MessageId: 084da79b-4570-421c-95be-5f1c0a7f13da, DequeueCount: 1, InsertedOn: 2026-11-05T17:56:09.000+00:00
[2026-11-05T17:56:10.771Z] Received order message: {"orderId":"D3-ENC-001","sku":"SKU-501","qty":2}
[2026-11-05T17:56:10.773Z] Order D3-ENC-001 processed successfully. SKU=SKU-501, Qty=2
[2026-11-05T17:56:10.802Z] Executed 'Functions.OrderQueueProcessor' (Succeeded, Id=1d51098d-aaa0-4115-92d1-75349215914d, Duration=220ms)
```

Step 8 – Send a malformed message

`dotnet run -- malformed`

Expected:

- The message is decoded successfully.
- JSON parsing fails.
- The Function invocation fails.
- The host retries the message.

Because `maxDequeueCount` is set to 3, the message is moved to `orders-poison` after three failed processing attempts.

```
PS C:\Users\LabsKraft\Day3-QueueLab\Day3-QueueProducer> dotnet run -- malformed
Malformed message sent.
PS C:\Users\LabsKraft\Day3-QueueLab\Day3-QueueProducer>
```

```
[2026-11-05T17:58:37.543Z] Message has reached MaxDequeueCount of 3. Moving message to queue 'orders-poison'.
[2026-11-05T17:58:37.640Z] Executing 'Functions.OrdersPoisonHandler' (Reason='New queue message detected on 'orders-poison'.', Id=0797d621-4245-4fb6-831b-6e24ba453bee)
[2026-11-05T17:58:37.642Z] Trigger Details: MessageId: 1c40818a-61b2-47c6-8ba3-4155189560db, DequeueCount: 1, InsertedOn: 2026-11-05T17:58:37.000+00:00
```

Step 9 – Create the poison-message handler

Create `PoisonOrderHandler.cs`:

```
using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

namespace ContosoFn;

public class PoisonOrderHandler
{
    private readonly ILogger<PoisonOrderHandler>
_logger;

    public PoisonOrderHandler(
        ILogger<PoisonOrderHandler> logger)
    {
        _logger = logger;
    }

    [Function("PoisonOrderHandler")]
    public void Run(
        [QueueTrigger(
            "orders-poison",
            Connection = "AzureWebJobsStorage")] string
message)
    {
        _logger.LogError(
            "POISON MESSAGE RECEIVED. Body: {Message}",
            message);
    }
}
```

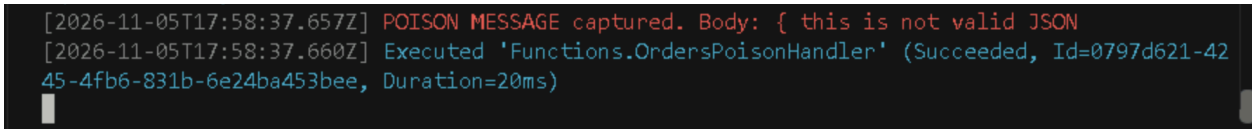
Build and restart the Function:

```
dotnet build
func start
```

Verify the `orders-poison` queue using Storage Explorer.

Expected:

- The malformed message is present in `orders-poison`.
- The poison handler is invoked.
- The malformed body is logged at error level.
- Valid messages continue to process successfully.



Step 10 – Deploy and validate in Azure

Publish the Function App:

```
func azure functionapp publish fn-user01-orders-d3
```

Ensure the Function App uses the Day 3 Azure Storage Account and has the required Queue data permissions. For managed identity authentication, configure the required Storage Queue data role and supported connection settings before relying on identity-based access.

```
PS C:\Users\LabsKraft\Day3-QueueLab\function-app> func azure functionapp publish fn-user01-ord
ers-d3
Time Elapsed 00:00:23.58

Getting site publishing info...
[2026-11-05T18:05:39.351Z] Starting the function app deployment...
Creating archive for current directory...
Uploading 3.77 MB [##############################################################################################]
Upload completed successfully.
Deployment completed successfully.
[2026-11-05T18:07:32.007Z] Syncing triggers...
Functions in fn-user01-orders-d3:
    OrderQueueProcessor - [queueTrigger]

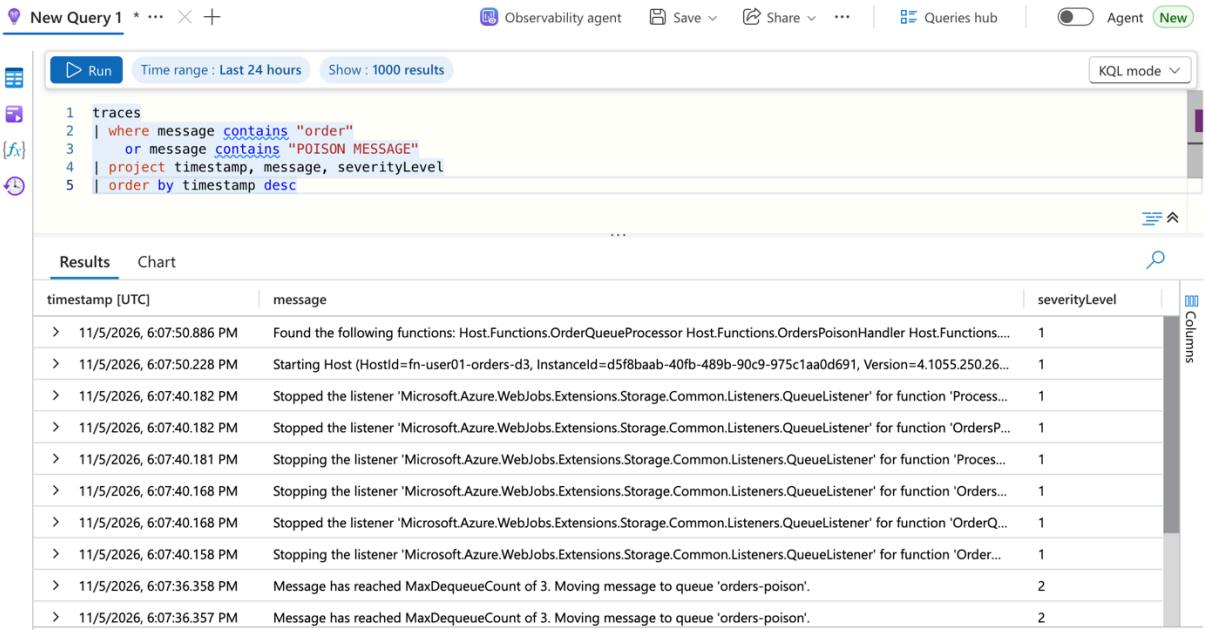
    OrdersPoisonHandler - [queueTrigger]

    ProcessOrder - [queueTrigger]

PS C:\Users\LabsKraft\Day3-QueueLab\function-app>
```

Use Application Insights to inspect failures and successful processing:

```
traces
| where message contains "order"
    or message contains "POISON MESSAGE"
| project timestamp, message, severityLevel
| order by timestamp desc
```



Acceptance criteria

Test	Expected result
Plain-text message	Encoding failure reproduced
Base64 message	Successfully processed
Malformed JSON	Processing fails
Retry configuration	Three processing attempts
Poison queue	Message appears in `orders-poison`
Poison handler	Logs message body at error level
Valid orders	Continue to process successfully

Production takeaway: Configure retry limits deliberately, monitor failed messages and create a poison-message handling process. Do not allow invalid messages to retry forever. Make consumers idempotent because message delivery can occur more than once.